

TASK RECORD

NAME: VIKASINI.K

DEPARTMENT: B. Tech CSE (Artificial Intelligence and Data Science)

COLLEGE: Vels institute of science, Technology and Advanced studies

INTERNSHIP ROLE: Data Analyst

TASK: SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

Task Description:

The objective of this task is to understand the **Software Development Life Cycle (SDLC)** and its different phases involved in developing software. The task focuses on studying how a software project progresses from the initial idea and requirement gathering to design, development, testing, deployment, and maintenance.

INTRODUCTION:

Software Development Life Cycle (SDLC) is a systematic process used to design, develop, test, deploy, and maintain software applications. It provides a structured approach for developing software from the initial idea to the final product.

I understood that SDLC is important because software development cannot be done effectively by simply writing code immediately. A proper development process helps the team understand the requirements, plan the project, design the system, develop the software, test it, deploy it, and maintain it.

The main purpose of SDLC is to develop high-quality software that satisfies user requirements while controlling time, cost, and resources.

MAIN PHASES OF SDLC:

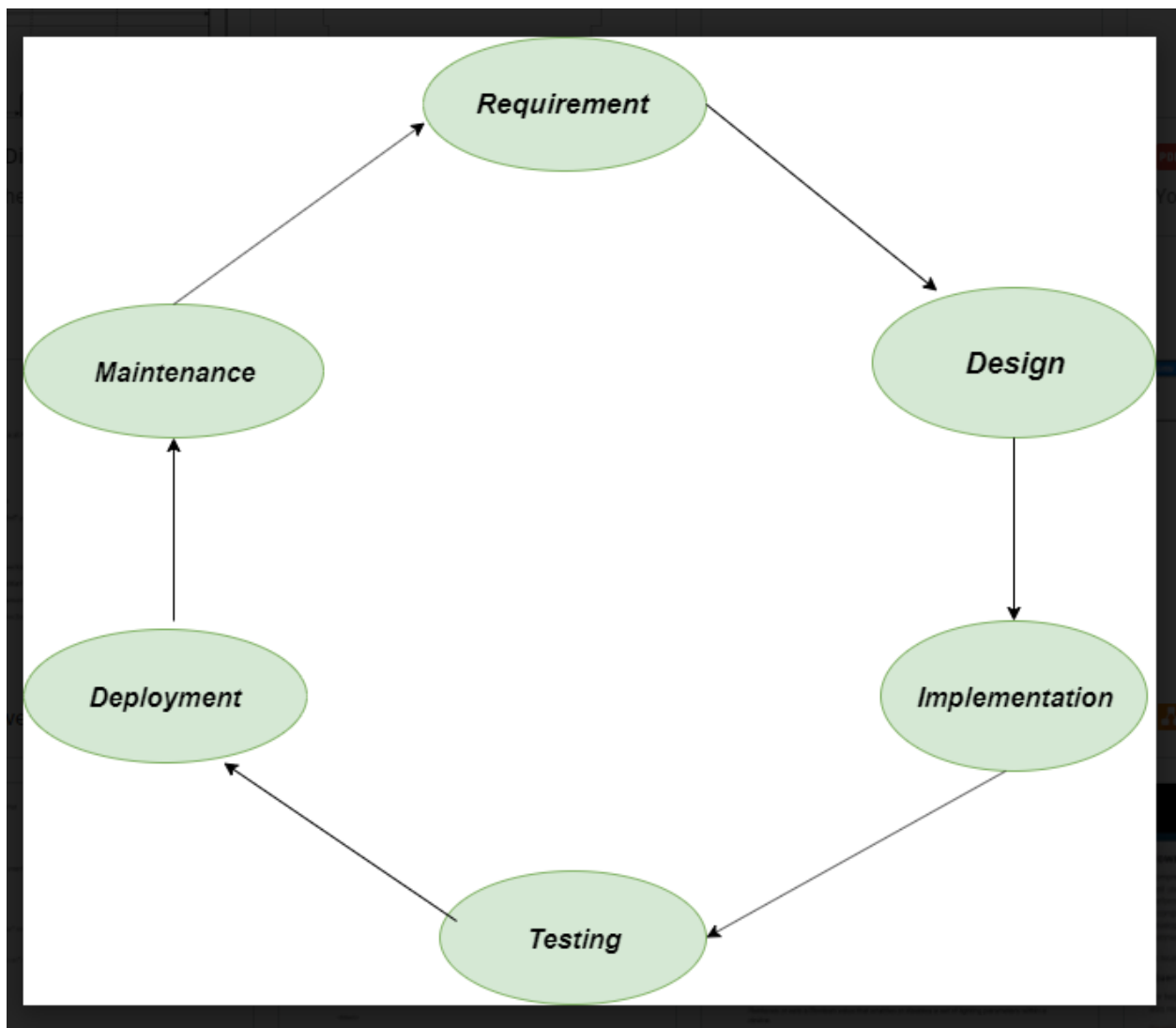
The Software Development Life Cycle generally consists of the following phases:

1. Planning
2. Requirement Analysis
3. System Design

4. Development / Implementation
5. Testing
6. Deployment
7. Maintenance

The phases are connected to each other, and feedback from later stages can sometimes result in improvements or changes to earlier stages.

STRUCTURE:



1. PLANNING:

Planning is the first major phase of SDLC.

In this phase, the development team identifies the purpose of the project and determines what needs to be developed. The team considers factors such as project scope, time, cost, resources, technology, and possible risks.

What I understood:

I understood that planning provides a clear direction for the entire project. Before starting development, the team should know:

- What problem the software will solve
- Who will use the software
- What resources are required
- How much time the project may take
- What technologies may be used
- What risks may occur

Example

For a student attendance application, the planning stage would identify students, faculty, attendance management, reports, and login as important areas of the system.

2. REQUIREMENT ANALYSIS:

Requirement analysis is the phase where the development team identifies and understands what the users need from the software.

Requirements are generally divided into:

Functional Requirements:

These describe what the system should do.

Examples:

- User should be able to register.
- User should be able to log in.
- Admin should be able to add records.
- The system should generate reports.

NON-FUNCTIONAL REQUIREMENTS:

These describe how the system should perform.

Examples:

- The application should be secure.
- The system should respond quickly.
- The application should be easy to use.
- The system should be reliable.

What I understood:

I understood that requirement analysis is one of the most important stages because incorrect or incomplete requirements can cause problems later in development.

The development team should communicate with users or customers and clearly document their requirements before designing and developing the system.

3. SYSTEM DESIGN:

After understanding the requirements, the team creates the design of the software.

System design describes how the software will work internally.

It may include:

- System architecture
- Database design
- User interface design
- Module design
- Data flow

- Technology selection
- Security design

What I understood:

I understood that design acts as a blueprint for the software.

For example,

if we are developing an online shopping application, the design may contain modules such as:

- User Management
- Product Management
- Shopping Cart
- Payment
- Order Management
- Database

The design helps developers understand how these components will communicate with each other before actual coding begins.

4. DEVELOPMENT / IMPLEMENTATION:

Development is the phase where the actual software is created.

Developers convert the system design and requirements into source code using suitable programming languages and technologies.

For example:

- Python
- Java
- JavaScript
- HTML and CSS
- SQL

Developers may divide the project into smaller modules and implement each module separately.

What I understood:

I understood that coding is only one phase of the complete software development process. Good coding should follow the requirements and design prepared in the earlier phases.

Version control tools such as Git can also be used to track changes made to the source code and allow developers to collaborate effectively.

5. TESTING:

After development, the software must be tested to identify errors, bugs, and unexpected behaviour.

Testing ensures that the software works according to the requirements.

Common types of testing include:

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing
- Regression Testing

What I understood:

I understood that testing is not simply about finding errors. It is also about checking whether the software satisfies the user's requirements.

For example, in a login system, testers may check:

- Correct username and password
- Incorrect password
- Empty fields
- Invalid username
- Password security
- Multiple login attempts

Finding and fixing bugs before deployment improves the quality and reliability of the software.

6. DEPLOYMENT:

After the software has been successfully tested, it is released to the users.

This process is called deployment.

Deployment can involve:

- Installing the application
- Configuring servers
- Setting up databases
- Publishing a website
- Releasing a mobile application
- Making the software available to customers

What I understood:

I understood that deployment is the stage where the developed software moves from the development/testing environment into the real-world environment.

For example, a website developed locally on a developer's computer can be deployed to a web server so that users can access it through the internet.

7. MAINTENANCE:

Software development does not end after deployment.

Once users start using the application, new problems, requirements, and improvements may arise. Therefore, the software needs continuous maintenance.

Maintenance may include:

- Fixing bugs
- Improving performance
- Adding new features
- Updating security
- Updating technologies
- Modifying the software according to user requirements

What I Understood:

I understood that maintenance is an important long-term phase of SDLC. Real-world software continuously changes based on user feedback, business requirements, security issues, and technological changes.

SDLC FLOW:

The overall SDLC process can be represented as:

Planning → Requirement Analysis → Design → Development → Testing → Deployment → Maintenance

The process may not always be strictly one-directional. If a problem is identified during testing, developers may need to return to the development or design stage. Similarly, changes in requirements may require modifications to the design and implementation.

IMPORTANCE OF SDLC:

I understood that SDLC is important because it:

- Provides a structured development process
- Helps understand user requirements
- Reduces development risks
- Improves software quality
- Helps manage project time and cost
- Makes development easier to organize
- Helps identify bugs early
- Improves communication between team members
- Makes software maintenance easier
- Helps deliver software that meets user expectations

EXAMPLE OF SDLC IN A REAL PROJECT:

- Consider an Online Food Ordering System.
- Planning
- Identify the need for an online platform where customers can order food.
- Requirement Analysis
- Identify requirements such as customer registration, restaurant listing, food selection, cart, payment, and order tracking.
- Design
- Design the application interface, database, system architecture, and different modules.
- Development
- Develop the frontend, backend, database, login system, order system, and payment functionality.
- Testing
- Test login, food selection, cart operations, payment, and order processing.
- Deployment
- Deploy the application so customers can access and use it.
- Maintenance
- Fix bugs, improve performance, add new features, and update the application based on customer feedback.
- This example helped me understand that SDLC is not just a theoretical concept. It is a practical process followed while developing real-world software.

My Understanding of SDLC:

From studying SDLC, I understood that successful software development requires more than programming knowledge. A developer needs to understand the complete software development process.

Each phase has a specific purpose:

- Planning tells us what we are going to build.
- Requirement Analysis tells us what the users need.
- Design tells us how the system will be structured.
- Development converts the design into working software.
- Testing checks whether the software works correctly.
- Deployment makes the software available to users.
- Maintenance keeps the software useful and reliable after release.

I also understood that mistakes in the early stages can affect the later stages. Therefore, proper planning, clear requirements, good design, and continuous testing are important for successful software development.

CONCLUSION:

Software Development Life Cycle provides a systematic approach to software development. It helps development teams move from an initial idea to a working and maintainable software product.

Through understanding SDLC, I learned that software development is a continuous process involving planning, analysis, designing, coding, testing, deployment, and maintenance. Each phase contributes to the quality of the final product.

Overall, SDLC helps developers build software in a more organized, efficient, reliable, and user-focused manner.